

- [17] BOURBAKI N. Worse is better is worse [EB/OL]. [2022-12-06]. <https://www.dreamsongs.com/Files/worse-is-worse.pdf>.

操作系统结构：扫码反馈



硬件环境与软件抽象

与用户态的应用程序类似，操作系统本身也是一个程序：两者都由大量指令组成，运行在由处理器、内存和各种设备组成的硬件环境中。与应用程序不同的是，操作系统具有更高的权限，可以支配所有的硬件资源；而低权限的应用程序往往只能使用部分硬件资源。为此，处理器为操作系统提供了专门的特权运行环境，使得操作系统除了可以像应用程序一样对硬件资源进行普通操作之外，还可以进行特权操作，从而有能力管理不同的应用程序，同时也有机会为应用程序提供硬件资源的抽象，并通过系统调用的方式向应用提供服务，如图 3.1 所示。在本章中，我们将介绍应用程序与操作系统的运行环境，硬件为操作系统提供的特权机制，以及操作系统利用这些机制为用户程序提供的抽象与接口。

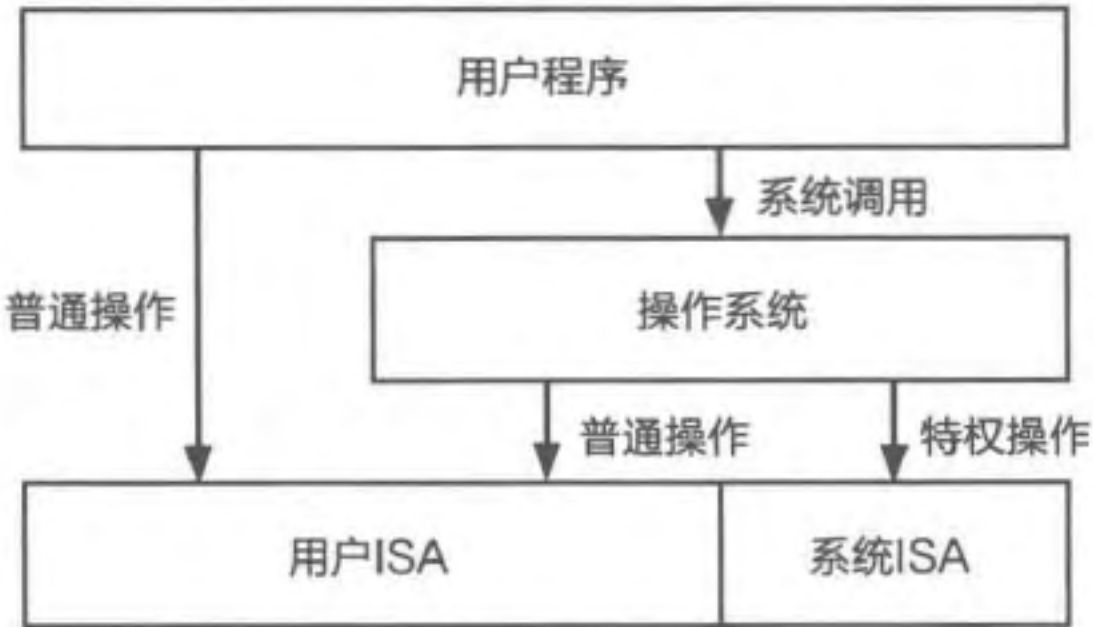


图 3.1 用户程序、操作系统、硬件环境之间的交互

3.1 应用程序的硬件运行环境

本节主要知识点

- 应用程序运行时依赖处理器中的哪些状态？
- 在函数调用和返回的过程中处理器做了哪些操作？

在介绍操作系统的具体实现之前，有必要先介绍硬件为应用程序提供的运行环境。操作系统的主要任务之一是管理应用程序，而当应用程序运行在硬件上时，硬件（主要是处理器）会保存诸多状态，比如，程序当前执行到哪条指令，上一条指令是否发生溢

出，函数调用依赖的栈在内存中的什么位置，等等。在应用程序的启动、切换以及退出阶段，操作系统需要对这些保存在硬件上的状态进行初始化、保存/恢复、清除等操作。本节首先介绍应用程序在处理器上的运行方式，以及一些与编译、ARM 汇编相关的必要基础知识，然后归纳出应用程序在运行时保存在处理器中的状态。对本节内容比较熟悉的读者，可以直接阅读本节最后的小结作为回顾。

3.1.1 程序的运行：用指令序列控制处理器

对熟悉 C 语言等高级编程语言的程序员来说，将源代码构建可执行程序然后运行是一件非常自然的事情。但你是否思考过，为什么硬件不能直接运行 C 语言的源代码呢？同样一份源代码，在 Linux 下编译出的二进制程序，放到 Windows 上是否能运行呢？二进制程序在硬件中究竟是如何执行的呢？

从 C 语言代码到汇编指令再到机器指令

高级语言的表达能力是很强的，它允许程序员使用变量、循环、函数这些强大的抽象来构建需要的逻辑。如果能让处理器直接理解和执行高级语言代码，就需要在处理器内部实现解析这些高级语言语义的逻辑，这会带来难以想象的硬件复杂度，使得处理器的体积和设计难度显著增大。因此，硬件设计者选择让处理器支持一套格式相对固定、功能相对简单、通常采用二进制编码的机器指令（Machine Instruction，简称指令^①），然后通过机器指令的有机组合去实现高级语言中复杂的语义与功能。例如，C 语言中的一条 `return a + b` 语句包含两个操作：对两个数求和，以及将结果返回给调用者。这两个操作分别对应两条机器指令，其编码分别为 `0x2000000b` 和 `0xc0035fd6`，处理器通过解析指令编码来“读懂”指令的内容，从而执行相应的操作。

显然，为了便于机器理解而设计的指令并不适合程序员理解。为此，处理器设计者为每一条机器指令设计了与其唯一对应的汇编指令（Assembly Instruction），它使用文本格式以便程序员阅读和修改。图 3.2 展示了前文提到的 `return a + b` 语句翻译为汇编指令和机器指令的过程：`0x2000000b` 和 `0xc0035fd6` 两条机器指令所对应的汇编指令分别写作 `add w0, w1, w0` 和 `ret`。其中的 `w0` 和 `w1` 指示了变量 `a` 和 `b` 在处理器中存放的位置，以及它们相加的结果将要存放的位置；`ret` 则对应从当前函数返回的功能。

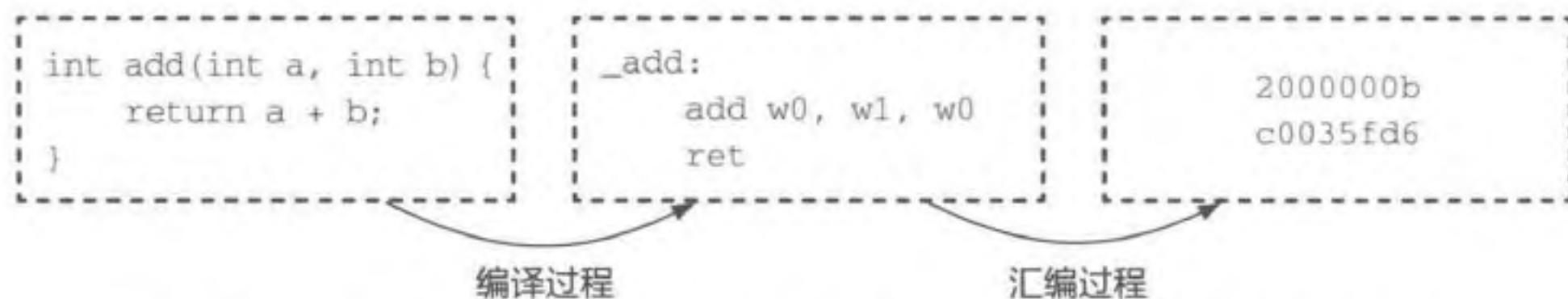


图 3.2 从 C 语言源程序编译为汇编程序，再汇编为机器程序的过程

① 本章中提到“指令”时如果不加前缀，均是指机器指令。

在通过源代码构建可执行程序的过程中，编译工具^①会首先将文本格式的高级语言源代码编译（Compile）为汇编指令的序列（即汇编程序），再将汇编指令序列汇编（Assemble）为机器指令的序列（即机器程序），并以二进制数据的形式保存在文件中，形成可执行的程序文件^②。使用不同的编译器或编译选项，可以把同一份高级语言源代码编译为多份不同但等效的汇编程序；但通常一份汇编程序只会被汇编为一份机器程序，因为汇编过程就是简单地将每条汇编指令翻译为对应的机器指令，不会发生程序结构上的变化。在默认情况下，编译器会将这两次翻译的细节隐藏起来，这样应用编程人员可以直接用一条命令（如 `gcc hello.c -o hello`）将高级语言源代码构建为可执行程序。

程序控制硬件的接口与规范：指令集架构（ISA）

应用程序对计算机硬件的控制，无论是让处理器进行加减运算，从内存读取数据或将数据写入内存，还是访问硬盘、网卡、键盘、显示器等外部设备，都是通过将相应的机器指令发送给处理器执行来实现的。关于这些机器指令的格式、行为及处理器在执行中的状态有一个规范，称为处理器的指令集架构（Instruction Set Architecture, ISA）。换句话说，ISA 就是处理器向软件提供的接口。通过将处理器的实现与接口解耦，程序员只需要面向 ISA 进行编程，而不用关注处理器硬件的实现细节；处理器制造商只需要照着 ISA 实现，而不用关心上层运行什么软件。例如，华为海思与高通公司在设计处理器时使用的都是 ARM 的 ISA，因此从软件的角度来看，这两家公司生产的处理器在执行同一条指令时的行为是一样的。

程序计数器与指令的执行顺序

处理器运行应用程序的流程非常简单：首先从可执行程序中取出第一条指令，根据 ISA 的规范解析这条指令并执行相应的操作，然后取出下一条指令并重复上述过程。这就引出了一个问题：每次执行完一条指令后，处理器如何知道接下来应该执行哪一条指令？主要有两种方式：顺序执行和跳转执行。

处理器内部有一个程序计数器（Program Counter, PC），专门用于记录即将执行的下一条指令在程序中的位置。在应用程序执行之前，操作系统会先将其可执行文件加载到内存中，然后将 PC 的值设置为可执行文件中第一条指令所在的内存位置。接下来，当处理器从 PC 指向的内存位置取出并解析一条指令后，会将该指令的长度加到当前 PC 的值上，使 PC 指向下一条指令。这样一来，程序的指令就会按顺序一条接一条地被处理器取出、解析并执行。这种执行模式称为指令的顺序执行，如图 3.3 所示。

然而，仅有顺序执行并不足以支撑高级语言中的复杂控制逻辑。例如，C 语言中的条件语句 `if` 会根据条件是否满足来选择执行 `if` 后的逻辑还是 `else` 后的逻辑。在顺序

① 此处的“编译工具”指编译器驱动程序（例如 `gcc`），包含编译和汇编等。

② 可执行的程序文件中除了包含二进制机器指令序列外，还包含程序中使用的数据、高级语言符号表等内容。编译器将这些内容以特定的格式组织成可执行文件，以使它们能够被操作系统识别和执行。

执行模式中，处理器的下一条指令永远是确定的；然而，在执行条件语句时，处理器在进行条件判断后所执行的下一条指令存在两种可能，这意味着至少有一种情况需要用顺序执行以外的方式去实现。为此，ISA 提供了一种将 PC 值修改为由程序指定的目标位置的方式，称为控制流^①跳转，又称跳转执行，如图 3.3 所示。跳转执行允许程序根据需要调整其指令执行的顺序，从而支撑高级语言中分支、循环等语义的实现。

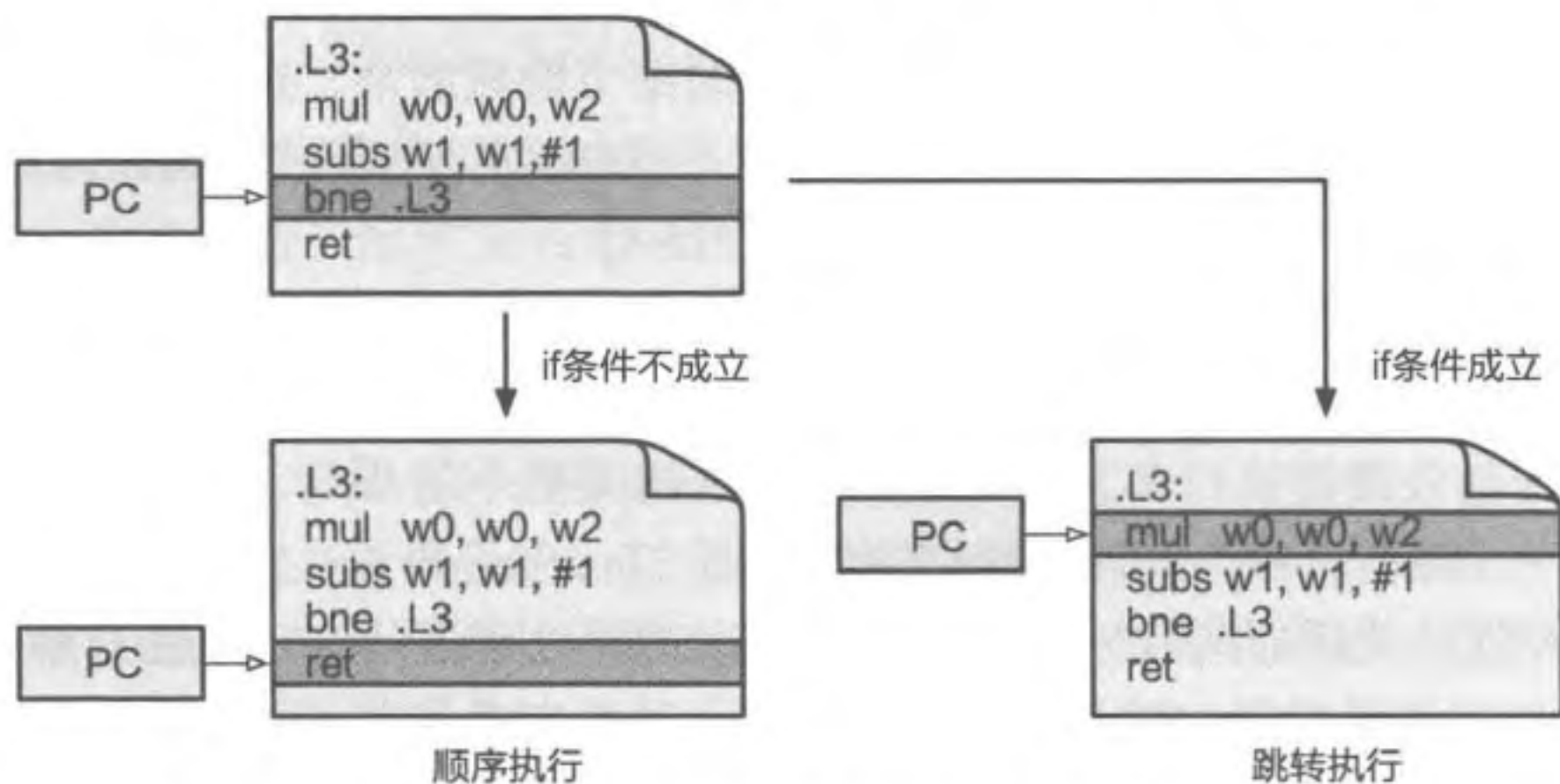


图 3.3 程序计数器值的两种常见更新方式：顺序执行与跳转执行

练习

3.1 仅仅依靠顺序执行和跳转执行两种方式，能否实现所有的分支和循环类型？如果能，请给出实现方式；如果不能，请举出反例。

3.1.2 处理数据：寄存器、运算和访存

数据处理，包括对数据的加、减、乘、除、与、或、异或、移位等操作，是处理器提供的基础功能。本节将关注与数据处理相关的硬件状态，包括用于暂存数据的寄存器 (Register) 和与处理器通过总线相连的内存 (Memory)。

寄存器：处理器内部的高速存储单元

寄存器是处理器内部的存储单元。相比内存，寄存器的主要特点是访问速度快、容量小。从应用的角度来看，寄存器主要可以分为两大类。一类是可以存储任意数据的寄存器，称为通用寄存器 (General-Purpose Register)。一般来说，处理器内部包含多个通用寄存器，例如 AArch64 架构有 31 个通用寄存器，如图 3.4 所示。对于不同的汇编指

^① 本章所讨论的控制流是指机器指令或汇编指令被执行的顺序，而不是像高级语言中那样指基本块被执行的顺序。

令，通用寄存器的用法可能不同：在进行运算时，它们可以用来存储源数据或运算结果；在读写内存时，它们可以用来存储要访问的内存位置；在调用函数时，它们可以用来传递参数。另一类是特殊寄存器，用来保存一些特定的数据，比如下一条执行指令的内存位置、栈顶的位置、条件码等，我们在后面的章节中会一一介绍。由于这两类之外的寄存器（例如用于存放浮点数的 128 位浮点寄存器）与操作系统相关性不强，本书不会对它们加以介绍。

处理器内部的数据运算和移动

程序员用 C 语言编程时，可以用一行表达式来完成复杂的计算，并用计算结果为某个变量赋值。汇编指令没有如此强大的表达能力，而是会将表达式拆解成移位、按位异或和乘法这样的基本运算，每个基本运算都由一条数据处理指令完成。常用的数据处理指令主要包括算术运算、逻辑运算、移位和数据搬移等类型，这类指令使用寄存器中存储的源数据进行算术或逻辑运算，并将结果存储到另一个寄存器中。例如，`add w0, w0, w1` 指令表示将 `w0` 和 `w1` 两个寄存器中的值相加，计算结果存放回 `w0` 寄存器。表 3.1 按照上述类型列出了一些 AArch64 的数据处理指令。

表 3.1 AArch64 指令集中一些常见的数据处理指令。数据处理指令会用第一个操作数指定存放结果的寄存器（即目的寄存器 `Rd`，`R` 表示 Register，`d` 表示 destination），用后面的操作数指定数据来源。有些数据来源是寄存器（用 `Rn` 表示），有些未必是寄存器（这里用 `Op2` 表示）。对寄存器以外的数据来源感兴趣的读者可以参考下文中的“小知识：立即数与修改过的寄存器”

指令类型	指 令	效 果	指令描述
算术运算指令	<code>add Rd, Rn, Op2</code>	$Rd \leftarrow Rn + Op2$	加法运算
	<code>sub Rd, Rn, Op2</code>	$Rd \leftarrow Rn - Op2$	减法运算
	<code>mul Rd, Rn, Op2</code>	$Rd \leftarrow Rn * Op2$	无符号乘法运算
	<code>div Rd, Rn, Op2</code>	$Rd \leftarrow Rn / Op2$	无符号除法运算
	<code>neg Rd, Rn</code>	$Rd \leftarrow -Rn$	取相反数



图 3.4 AArch64 包括 31 个 64 位的通用寄存器，编号为 0 ~ 30。标号中的前缀代表指令要使用寄存器中的哪一部分数据，其中 `X` 表示取完整的 64 位，`W` 表示取低 32 位。图 3.2 中的 `add w0, w1, w0` 指令使用的就是 0 号和 1 号寄存器的低 32 位，因为 C 语言源代码中参与计算的数据类型是 32 位整型

(续)

指令类型	指 令	效 果	指令描述
逻辑运算指令	and Rd,Rn,Op2	$Rd \leftarrow Rn \& Op2$	按位与
	orr Rd,Rn,Op2	$Rd \leftarrow Rn Op2$	按位或
	eor Rd,Rn,Op2	$Rd \leftarrow Rn \oplus Op2$	按位异或
	mvn Rd,Rn	$Rd \leftarrow \sim Rn$	按位取反
移位指令	asr Rd,Rn,Op2	$Rd \leftarrow Rn \gg_A Op2$	算术右移
	lsl Rd,Rn,Op2	$Rd \leftarrow Rn \ll Op2$	逻辑左移
	lsr Rd,Rn,Op2	$Rd \leftarrow Rn \gg_L Op2$	逻辑右移
	ror Rd,Rn,Op2	$Rd \leftarrow Rn \gg_R Op2$	循环右移
数据搬移指令	mov Rd,Op2	$Rd \leftarrow Op2$	数据移动

接下来，我们将以一个哈希函数（代码片段 3.1）为例来熟悉处理器提供给可执行程序的数据处理接口。在这段代码中，C 语言源代码中共有五个算术操作，包括重复了两遍的右移和异或操作，以及一个加法操作。对应到汇编代码中，内容相同的第 2 行和第 5 行都是同时完成了右移（asr）和异或（eor）操作，而第 3 行将加法操作的操作数加载到寄存器，再由第 4 行完成加法（add）操作。

代码片段 3.1 哈希函数的 C 语言代码及对应的汇编代码

<pre>1 int hash(int src) 2 { 3 src = ((src >> 16) ^ src) + 0xbeef; 4 return (src >> 16) ^ src; 5 }</pre>	
<pre>1 hash : 2 eor w0, w0, w0, asr 16 3 mov w1, 48879 4 add w0, w0, w1 5 eor w0, w0, w0, asr 16 6 ret</pre>	

小知识：立即数与修改过的寄存器

源数据中的 Op2 不一定是寄存器。例如，mov w1, 48879 中的源数据就是一个立即数（Immediate）[⊖]，在汇编语言中用于表示常量。eor w0, w0, w0, asr 16 中的 w0, asr 16 部分在 AArch64 架构中被称为修改过的寄存器（Modified Register），它代表将 w0 寄存器的值向右移 16 位后的结果作为 eor 指令的源数据之

⊖ AArch64 汇编程序中的立即数可写为一个单独的数值，也可写为一个“#”符号后跟一个数值。编译器生成的汇编代码采用哪种方式视编译器实现而定，两者均是合法的。

一。编译器在生成汇编代码时常常会运用修改过的寄存器来减少指令数量和用于存放中间结果的寄存器占用量，以优化程序的运行效率。

练习

3.2 假设上例中的哈希函数开始执行时 w0 寄存器的值为 0x87654321。
(a) 请根据表 3.1 中的指令格式与功能介绍，写出函数执行每条指令后 w0 寄存器的值。

汇编指令	执行后 w0 寄存器的值
eor w0, w0, w0, asr 16	
mov w1, 48879	
add w0, w0, w1	
eor w0, w0, w0, asr 16	

(b) 思考一下，如果将上表中使用的寄存器全部换成 x0 和 x1，这些指令还会得到相同的执行结果吗？为什么？

处理器视角下的内存

在执行程序的过程中，处理器往往需要与内存进行数据交换。程序中的绝大部分数据都是存放在内存中的，处理器在进行运算之前需要先将数据从内存加载到寄存器，计算完成后再将结果从寄存器写回内存。因此，ISA 需要提供内存的加载指令与存储指令，分别实现内存的读取与写入功能，合称为访存指令。表 3.2 列出了 AArch64 指令集中的访存指令。接下来我们将通过 swap 函数（用于交换两个内存位置中的数据，见代码片段 3.2）来介绍它们的使用方式。

表 3.2 AArch64 指令集中的访存指令。表中 R_s 指寄存器的大小，mem[a : b] 指地址 a 到地址 b 的内存范围

指令类型	指 令	效 果	指令描述
加载指令	ldr R, addr	$R \leftarrow \text{mem}[\text{addr} : \text{addr} + R_s]$	从内存加载数据到寄存器
	ldp R1, R2, addr	$R1, R2 \leftarrow \text{mem}[\text{addr} : \text{addr} + R1_s + R2_s]$	从内存加载数据到两个寄存器
存储指令	str R, addr	$R \rightarrow \text{mem}[\text{addr} : \text{addr} + R_s]$	将寄存器中的数据存储到内存
	stp R1, R2, addr	$R1, R2 \rightarrow \text{mem}[\text{addr} : \text{addr} + R1_s + R2_s]$	将两个寄存器中的数据存储到内存

代码片段 3.2 swap 函数的 C 语言代码及对应的汇编代码

```
1 void swap (int* a, int* b)
2 {
```



```
3  int temp = *a;
4  *a = *b;
5  *b = temp;
6 }
```

```
1 swap:
2  ldr w3, [x1]
3  ldr w2, [x0]
4  str w3, [x0]
5  str w2, [x1]
6  ret
```

代码片段 3.2 的汇编部分用两个阶段完成了数据位置的交换：第一个阶段（第 2、3 行）使用加载指令 `ldr` 从内存中读取 `a` 和 `b` 两个指针指向的值，第二个阶段（第 4、5 行）使用存储指令 `str` 将刚才读取的两个值以相反的顺序放回内存中 `a` 和 `b` 所指向的位置。可以知道，指令中的 `w2` 和 `w3` 是寄存器，用于存放从内存中读出的数据，但 `[x0]` 和 `[x1]` 是什么意思呢？

从处理器的视角来看，内存就是一个很大的数组，数组中的每个元素为 1 字节，数组的索引称为内存地址（Memory Address）。在使用加载指令和存储指令访问内存时，指令需要指定所访问的内存地址；处理器解析指令得到地址，并根据地址在内存中找到对应的数据，这个过程称为寻址（Addressing）。寻址可以有不同的模式，在 AArch64 架构中，寻址主要包括两种模式，分别称作偏移量寻址（Offset Addressing）与索引寻址（Index Addressing）。两种寻址方式都涉及基地址（base）和偏移量（offset）。偏移量寻址的格式为 `[base, offset]`，以基地址和偏移量之和作为目标内存地址。索引寻址则可以进一步分为前索引寻址（格式为 `[base, offset]!`）和后索引寻址（格式为 `[base], offset`），两者都将基地址作为目标内存地址，并分别在寻址操作前后将基地址的值更新为基地址与偏移量之和。上面例子中的 `[x0]` 和 `[x1]` 使用的就是偏移量寻址，这里的 `x0` 和 `x1` 寄存器的值被作为基地址，偏移量则被省略了。在 3.1.4 节中，我们会接触到索引寻址在函数中的应用。

练习

3.3 假设内存区域 `0x1000 ~ 0x1008` 及通用寄存器 `x0` 和 `x1` 的内容如下（字节序为小端序）。

内存地址	内存地址的值	寄存器	寄存器的值
0x1000	0x01234567	x0	0x1000
0x1004	0x89abcdef	x1	0x1

请计算出下表中每条指令执行后相关寄存器或内存位置的值（指令之间没有相互影响）。

指令	位置	值
ldr x2, [x0]	x2 寄存器	
ldr w2, [x0, #2]	x2 寄存器	
str w1, [x0, x1, lsl #2]	内存区域 0x1000 ~ 0x1008	
stp w0, w1, [x0]	内存区域 0x1000 ~ 0x1008	

3.1.3 条件结构：程序分支和条件码

在介绍程序的运行时，我们曾介绍过指令的两种执行模式：顺序执行和跳转执行。其中，顺序执行是处理器默认的执行模式，而跳转执行主要是为了与条件判断相结合以实现“有条件的代码执行”。在 ISA 的支持下，程序可以通过判断一个条件是否满足来决定是否需要跳转到另一条指令的位置，从而实现条件分支和循环等复杂程序设计中不可或缺的结构。这牵涉两个重要功能：跳转到目标代码位置，以及根据条件判断决定是否执行下一条指令。接下来，我们以一个幂函数（见代码片段 3.3）为例分别讲解 ISA 对这两个功能的实现，即程序分支与条件码。

代码片段 3.3 幂函数的 C 语言代码及对应的汇编代码

<pre>1 int power(int x, unsigned int n) 2 { 3 int result = 1; 4 for (unsigned int i = n; i > 0; i--) 5 result *= x; 6 return result; 7 }</pre>	
<pre>1 power: 2 mov w2, w0 3 mov w0, 1 4 cbz w1, .L1 5 .L3: 6 mul w0, w0, w2 7 subs w1, w1, #1 8 bne .L3 9 .L1: 10 ret</pre>	

分支目标与分支指令

我们在观察幂函数的汇编代码时可能会首先注意到，在函数名 power 之外还有两个

标签 (Label), 分别是 “.L3” 和 “.L1”。这些标签在汇编语言中专门用来定位某处汇编代码或数据的地址, 对它们的引用会在汇编程序到可执行程序的转变过程中被翻译为真实的地址, 而它们本身不会出现在可执行程序中。在我们的例子中, “.L3” 指示的是 `mul w0, w0, w2` 指令的地址, “.L1” 指示的是 `ret` 指令的地址。

有了用于定位的标签, 我们还需要能够跳转到标签位置的分支指令 (Branch Instruction)。例子中总共展示了两种分支指令。一种如 `bne .L3` 所示, 它根据前一条指令执行的状态 (如结果是否为零、计算过程中是否产生溢出) 来判断自己的条件是否满足, 若满足则跳转。在这里, 指令中的 “b” 代表 “Branch”, “ne” 代表 “Not Equal”, 其语义是 “当运算结果与零不相等时跳转”。另一种则如 `cbz w1, .L1` 所示, 根据本条指令内置计算的执行状态而非前一条指令的执行状态进行条件判断。指令中的 “c” 代表 “Compare”, “z” 代表 “Zero”, 因此该指令在 `w1` 的值为零时会跳转到 “.L1”, 否则继续顺序执行。

条件判断与条件分支

对于 `bne` 这种没有内置计算功能的指令而言, 上一条指令的执行状态是如何传达给它的呢? 一种简单的想法是为每一种可能的条件设置一个寄存器 (或某个寄存器中的一位), 并根据上一条指令的执行状态设置这些寄存器 (或位)。常见的 ISA 确实采用了类似的设计, 只不过会将所有复杂的条件表示为寥寥几个特征的组合, 这样就只需要记录每次条件计算是否表现出这些特征, 大大减少了条件计算的开销。这些特征统称为条件码 (Condition Flag), 一般会被实现为记录程序运行状态的状态寄存器 (如 x86 的 `FLAGS`、AArch64 的 `PSTATE`) 中的若干个位。虽然在不同架构中的命名有所区别, 但条件码一般只包含四位, 分别表示计算结果的正负、计算结果是否为零、计算是否产生进位或借位以及计算是否产生有符号溢出。在 AArch64 架构中, 这四个条件码分别称为 `N` (Negative)、`Z` (Zero)、`C` (Carry)、`V` (Overflow)。常见的分支条件及对应的条件码如表 3.3 所示。

表 3.3 常见的分支条件及对应的条件码

条 件	含 义	对应的条件码 (NZCV)
EQ	相等	<code>Z = 1</code>
NE	不等	<code>Z = 0</code>
MI	负数	<code>N = 1</code>
PL	非负数	<code>N = 0</code>
HI	无符号大于	<code>C = 1 且 Z = 0</code>
LO	无符号小于	<code>C = 0</code>
LS	无符号小于或等于	<code>C = 0 或 Z = 1</code>
GE	有符号大于或等于	<code>N = V</code>
LT	有符号小于	<code>N != V</code>
GT	有符号大于	<code>Z = 0 且 N = V</code>
LE	有符号小于或等于	<code>Z = 1 或 N != V</code>

小知识：条件码的更新

细心的读者可能会注意到上例中进行减法运算的指令是 `subs`，和我们前面介绍的 `sub` 有所差异。事实上，这是为了根据减法运算的结果更新条件码。在 AArch64 架构中，算术逻辑运算指令只有加上 `s` 后缀才会更新条件码的值，这点与 x86 架构中绝大部分算术逻辑运算指令都会更新条件码的值有很大区别。

除了带有 `s` 后缀的算术逻辑运算指令外，AArch64 还支持 `cmp`、`cmn` 和 `tst` 三个专门的比较指令，它们分别根据操作数之差、操作数之和以及操作数相与的结果修改条件码。分支条件的设计是符合编程人员使用 `cmp` 指令时的直觉的。例如，在执行 `cmp x0, x1` 后使用 `bgt` 指令进行条件跳转，则会在 `x0` 寄存器中的值大于 `x1` 寄存器中的值时跳转。

下面我们来讨论条件分支指令是如何根据条件码的值决定是否需要跳转的。以代码片段 3.3 为例，编译器生成的 `subs w1, w1, #1` 指令既完成了 `i--` 的操作，又更新了条件码的值。由于 `i` 被声明为一个非负的无符号变量，C 语言源代码中 `i > 0` 的判定条件与“`i--` 的结果不为零”事实上是同时成立的，因此汇编代码可以通过判定 `subs w1, w1, #1` 指令得到的结果是否等于零来决定是否要离开循环。`bne .L3` 指令正是发挥了这个作用：NE 条件成立的条件是 `Z=0`，即上条指令（在这里是 `subs w1, w1, #1`）计算结果不为零。如果条件成立，说明 `i` 还没有减小到零，那么控制流会跳转回 `while` 循环的开头（`.L3`）并开始下一次循环；反之，说明循环条件已经不成立了，因此不进行跳转，控制流离开 `while` 循环。如果 `i` 是一个有符号变量的话，编译器会生成不同的跳转条件（例如 `bgt`）以明确地表达 `i > 0` 这一条件。

小知识：AArch64 中的分支指令

AArch64 架构中的分支指令可以分为两种：直接分支指令与间接分支指令。其中，直接分支指令以标签对应的地址作为跳转目标，又进一步分为无条件分支指令 `b` 和条件分支指令 `bcond`（`cond` 代表具体的条件，例如 `beq`，具体参见表 3.3）。间接分支指令 `br` 则以寄存器中的地址作为跳转目标。

练习

3.4 在幂函数的 C 语言代码中，我们可以观察到迭代器 `i` 的初始值被赋为 `n`。为什么在汇编代码中我们找不到为迭代器所在的 `w1` 赋初始值的指令？（请在完成下面两节有关函数的学习后回答这个问题。）

3.5 假设两个通用寄存器的值如下表所示。

X0	X1
5	-3

(a) 根据前面介绍的条件码定义和更新条件码指令的工作原理，计算出下表中每条指令执行后条件码的值。

指 令	N	Z	C	V
adds x0, x0, x1				
subs x0, x1, x0				
cmp x0, x1				
tst x0, x0				

(b) 根据前面介绍的分支条件定义，判断在执行 cmp x0, x1 后下表中的每条条件跳转指令是否会跳转。

指 令	是否跳转
beq	
bge	
bhi	
blo	

3.1.4 函数的调用、返回与栈

与前面介绍的计算、访存和跳转相比，高级语言中的函数这一抽象在汇编语言层面的实现更为复杂。首先，函数的调用与返回意味着控制流在调用者与被调用者之间的交接，ISA 一般会提供专门的函数调用指令与返回指令来完成这项任务。其次，函数在运行过程中需要占用部分内存来存放调用参数和局部变量等数据，ISA 采用运行时栈 (Runtime Stack) 为每个函数的实例分配可供临时使用的内存，并在函数实例返回后释放内存。接下来，我们将以一组简单的平方函数与立方函数（见代码片段 3.4）为例，详细介绍 ISA 为函数抽象提供的支持。

代码片段 3.4 平方函数与立方函数的 C 语言代码及对应的汇编代码

```
1 int square(int n)
2 {
3     return n * n;
4 }
5
```

```

6 int cube(int n)
7 {
8     return n * square(n);
9 }

```

```

1 square:
2     mul    w0, w0, w0
3     ret
4 cube:
5     stp    x29, x30, [sp, -32] !      # 前索引寻址
6     mov    x29, sp
7     str    x19, [sp, 16]
8     mov    w19, w0
9     bl     square
10    mul    w0, w0, w19
11    ldr     x19, [sp, 16]
12    ldp     x29, x30, [sp], 32        # 后索引寻址
13    ret

```

函数的调用指令与返回指令

函数的调用与返回过程涉及调用者与被调用者之间的控制流交接。具体而言，调用者需要知道被调用者的第一行代码地址，以跳转到被调用者的代码区域；被调用者也必须知道调用代码的位置，这样在执行完成后才能返回调用处，使调用者可以继续执行。前者相对容易，只要用一个标签表示函数名，前文提到的跳转执行就可以实现这一功能；后者比较复杂，因为同一个函数可能被不同的调用者调用，所以被调用者无法在实际被调用之前确定调用代码的位置。因此，处理器需要在发生函数调用时，将调用指令后的下一条指令地址保存在某个位置，从而在被调用者运行返回指令时，可以从该位置取出地址以继续执行——这个地址被称为返回地址（Return Address）。由此可见，函数调用指令需要完成两个任务：跳转到目标函数和保存返回地址。

尽管跳转的方式基本相同，但不同架构保存返回地址的方式却有着不小的差异。例如，x86 架构中的函数调用指令（call）会直接将返回地址保存到内存，返回指令（ret）则会从同一个内存位置读取返回地址，并跳转到要返回的代码处。而 AArch64 架构中的函数调用指令（bl）会将返回地址保存在一个特定的寄存器中，返回指令（ret）则会从该寄存器中读取地址并跳转过去。这个寄存器就是返回地址寄存器，在 AArch64 上实现为通用寄存器中的 X30 寄存器，别名为 LR（Link Register）。图 3.5 给出了一个例子，描述函数调用和返回过程中 PC 和 LR 寄存器的变化。AArch64 之所以如此实现，是因为 RISC 架构为了降低硬件的复杂度只允许专门的访存指令与内存进行数据交换，其他指令（包括函数调用指令）则只能访问处理器内部的寄存器。

在上面的例子中，如果被调用的函数在执行过程中又调用了其他函数，那么 LR 寄存器的值会被覆盖，这个问题该如何解决呢？事实上，编译器会在每个函数调用指令之

前和之后某处生成额外的代码，分别完成“将 LR 的旧值保存到内存”和“将内存中保存的值恢复到 LR”两个任务。注意，如果一个函数中没有对其他函数的调用（例如上例中的 square 函数），编译器就无须保存和恢复 LR 的值。与直接将返回地址保存到内存的 x86 架构相比，这种方式可以减少两次内存访问。

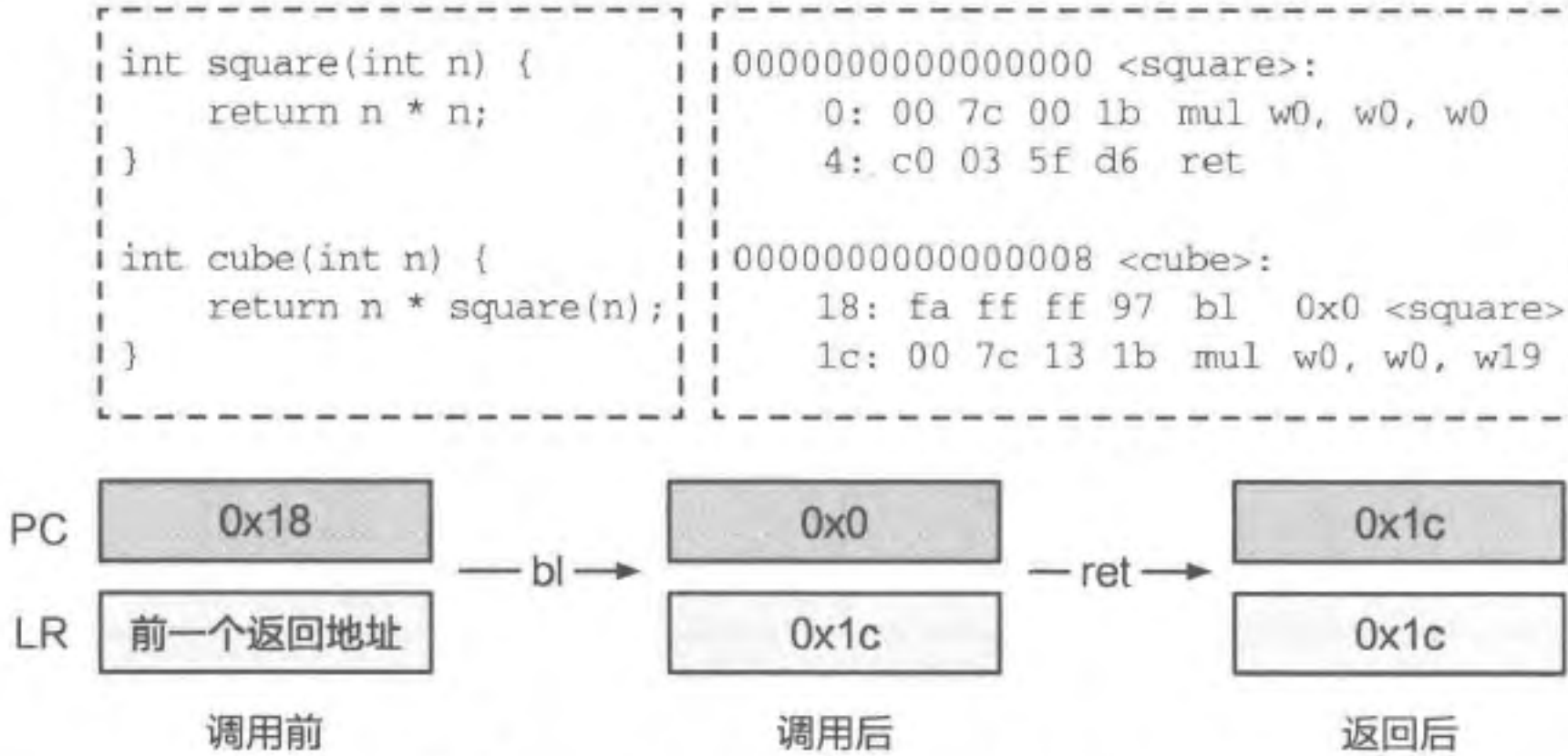


图 3.5 调用前后 PC 与 LR 值的变化

运行时栈：保存函数中的局部状态

每个函数在运行期间（从被调用到返回前）都需要一段内存来存放其局部状态，例如被调用时传入的实际参数、函数内声明的局部变量，等等。由于函数的调用可以嵌套，程序执行过程中往往存在多个未返回的函数，这些函数占用的内存区域被按照调用顺序排列在一起，新被调用的函数从这个序列的尾部申请内存，已经返回的函数释放自己占用的内存。考虑到函数调用的模式是“先被调用者后返回”，这些排列好的内存区域事实上可以被看作栈结构，其中先被调用的函数对应的内存区域更靠近栈底，也就更晚出栈（被释放）。我们将这个结构称为运行时栈或函数栈，在操作系统语境下一般简称“栈”。处理器一般会设置一个专门的栈寄存器来存放指向栈顶的指针——栈指针（Stack Pointer），它指示着当前运行时栈的大小：大于栈指针的地址是已经分配给某个函数的，而小于栈指针的地址是未分配的[Ⓐ]。在运行过程中，函数可以通过增大与减小栈指针的值分别在栈上分配与释放内存。

每个函数在栈上拥有的连续内存空间称为函数的栈帧（Stack Frame）。当前函数的栈帧在内存中的起始位置称为帧指针（Frame Pointer, FP），一般由一个寄存器来专门保存，如 AArch64 架构将 FP 保存在 X29 通用寄存器。编译器在生成函数的汇编代码时，会分别向函数的开头与末尾插入创建栈帧与释放栈帧的代码，从而保证生命周期已经结束的函数所占用的内存能够被正确地回收。图 3.6 展示了随着函数的调用与返回栈帧被创建与删除的机制。

Ⓐ 这是因为运行时栈一般从高地址向低地址增长。

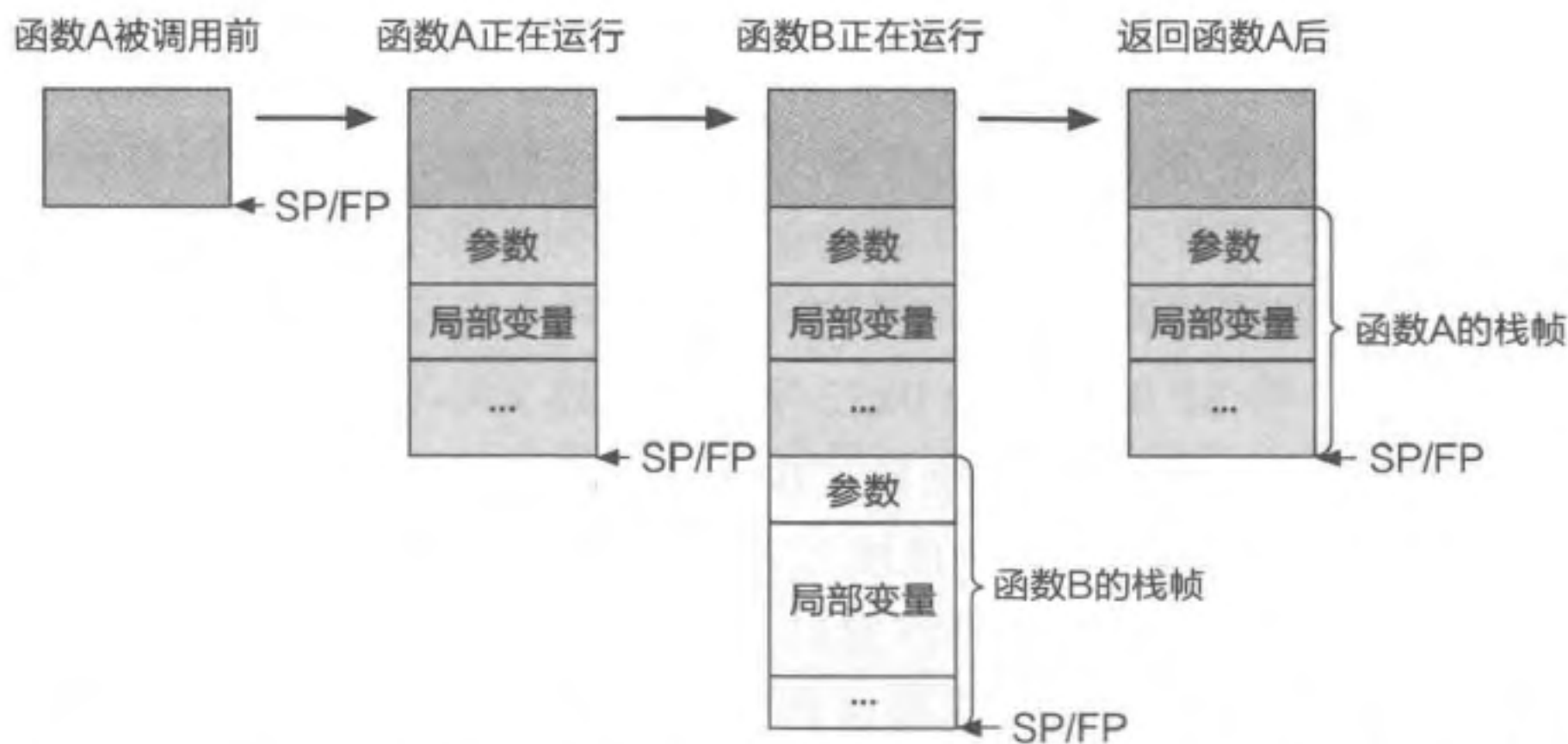


图 3.6 栈的内容在函数调用与返回过程中的变化，图中的函数 A 与函数 B 为调用关系。栈帧中每项内容的相对位置和被插入栈帧的时机由编译器决定，不同的编译器甚至同一编译器的不同版本可能会产生不同的栈帧结构

小知识：FP 的作用

如果我们知道了一个函数的栈帧位置，就可以通过偏移量寻址来访问这个函数管理的全部局部状态。由于每个调用者的 FP 都被保存在被调用者的栈帧中，FP 可以被用于实现栈追踪（Stack Trace），即从当前函数开始向前追溯出调用链中每个函数的栈帧，并获取栈帧中的局部状态。栈追踪在程序调试与漏洞修复中是非常重要的信息，本书的第一个实验包含一个简单的栈追踪实现，以加深读者对于运行时栈和调用惯例的理解。

考虑到保存和恢复 FP 需要额外的访存开销，而 FP 在程序的运行中又只起到帮助调试的作用，很多追求性能的项目会通过配置编译器来禁用 FP，也就是不再设置、保存与恢复 FP 的值。

小知识：重合的 FP 与 SP

细心的读者可能会注意到图 3.6 中的 FP 与 SP 重合了。事实上，这是编译器在栈上分配空间的方式所导致的。ARM 平台的编译器往往在函数开头直接将函数需要的所有栈内存分配完成，因此 SP 的值在函数执行过程中不会改变，与 FP 一般是重合的^①。x86 平台的编译器一般会使用入栈 / 出栈指令在函数执行过程中动态地分配和释放小段栈空间，SP 会随之改变，但 FP 始终指向栈帧的起始位置，SP 与 FP 之间的内存范围即当前函数的栈帧。

① ARM 平台编译器编译出的程序也存在 FP 与 SP 不重合的情况，例如利用栈传递参数时，压入栈中的参数不属于当前函数栈帧，处于 SP 与 FP 指示的内存地址之间。我们会在下一节进一步了解这种情况下栈的状态（图 3.7）。

由于 FP 和 LR 寄存器在处理器中都只有一份，因此每个函数都需要在覆盖这些值之前保存它们，并在自己的生命周期结束前（即返回前）恢复它们。AArch64 平台编译器生成的代码一般会借助前文介绍过的索引寻址，使得访存指令可以在保存或恢复 FP 和 LR 的值的同时减小或增大 SP 寄存器的值，从而用一条指令完成“FP/LR 值的保存与恢复”和“栈帧的创建与释放”两个任务。例如，cube 函数开头的代码 `stp x29, x30, [sp, -32]!` 将 SP 的值减小 0x32 字节（创建大小为 32 字节的栈帧），并将 FP 和 LR 保存到更新后的 SP 值指示的位置，也就是栈帧的低地址处。返回前的代码 `ldp x29, x30, [sp], 32` 则将 SP 的值增大 0x32 字节（释放栈帧），并从更新前的 SP 值指示的位置恢复 FP 和 LR 的值。值得注意的是，这两条指令在函数中的位置、数据在寄存器和内存间的流动方向以及对 SP 的修改都是完全对称的，这是为了在函数的生命周期结束后消除它对栈和寄存器状态的影响。

3.1.5 函数的调用惯例

在上一节中，我们探讨了控制流在调用者与被调用者间的交接过程，以及函数分配和释放临时内存的方式。事实上，调用者与被调用者之间需要交接的不仅是控制流，还有参数、返回值等数据。另外，由于调用者与被调用者共用同一组通用寄存器，它们还需要协调寄存器的分配，并在必要的时候对使用中出现冲突的寄存器进行保存和恢复。调用者和被调用者之间应当如何协调并不是由硬件的 ISA 决定的，而是由软件的调用惯例（Calling Convention）来规范的，并由编译器来实现。调用者与被调用者双方只有遵循调用惯例，才不至于在使用处理器时产生冲突，从而正确地完成数据的交接。

调用惯例是函数调用过程中调用者与被调用者双方需要遵循的规范。上一节的内容已经涉及调用惯例中栈帧的创建与回收、LR 和 FP 的保存与恢复等部分，本节则会探讨调用惯例中另外两个比较重要的约定：调用前后通用寄存器应当由谁来保存，以及参数与返回值如何在调用者与被调用者间传递。

寄存器保存：位置与方式

由于所有的函数都共享同一批通用寄存器，不同的函数对通用寄存器的使用难免发生冲突。为防止调用者存放在通用寄存器中的值被覆盖，调用惯例通常将通用寄存器划分成两部分，一部分由调用者保存（Caller-saved），另一部分由被调用者保存（Callee-saved）。具体而言，调用者需要在调用函数前保存一些通用寄存器的值，并允许被调用者随意使用这些寄存器；另一些通用寄存器的值则需要在调用前后保持一致，因此被调用者在修改这些寄存器的值之前需要先保存原值，并在返回前进行寄存器值的恢复。

在 AArch64 架构中，X9 ~ X15 由调用者保存，X19 ~ X28 由被调用者保存。上一节中展示的 cube 函数使用了一个由被调用者保存的寄存器（X19），因此在使用之前将它的值保存到栈帧中（`str x19, [sp, 16]`，这部分称为寄存器保存区），并在返回前进行了相应的恢复工作（`ldr x19, [sp, 16]`）。

练习

- 3.6 关于函数的调用惯例，请回答以下问题。
- (a) 调用者保存的寄存器和被调用者保存的寄存器中，哪一种适合存放生命周期较长、在整个函数内各处都会用到的变量？哪一种适合存放只是在一小段代码中临时用到的变量？
 - (b) 为什么不让调用者保存所有的寄存器，或让被调用者保存全部的寄存器？

数据传递：参数与返回值

函数调用期间，调用者和被调用者双方进行数据交换的方式主要有两种：调用者向被调用者传递参数，以及被调用者向调用者返回返回值。大部分处理器架构都会优先用寄存器完成参数和返回值的传递，例如 AArch64 架构中会分别用 X0 ~ X7 来传递前 8 个参数，并用 X0 传递返回值，如果有更多的参数则会保存到栈上（这部分称为参数构造区）。例如，在 cube 函数中，第一个参数 n 的值就放在 w0 中。由于 n 被直接作为参数传递给了 square 函数，第 9 行代码没有进行额外的参数准备就直接用 bl 指令调用了 square 函数。再观察 square 函数的代码，可以发现它将乘法的结果同样放在了 w0 中，作为返回值传递回 cube 函数。

小知识

在较低的优化等级下，AArch64 编译器会让每个函数在开始时将每个实际参数保存到栈帧中的参数保存区（见图 3.7），以保证这些参数的值在整个函数的生命周期内都可以被随时引用。为了减少内存访问带来的开销，高优化等级更倾向于使用被调用者保存的寄存器（X19 ~ X28）来保存参数，如 cube 函数中的 mov w19, w0。

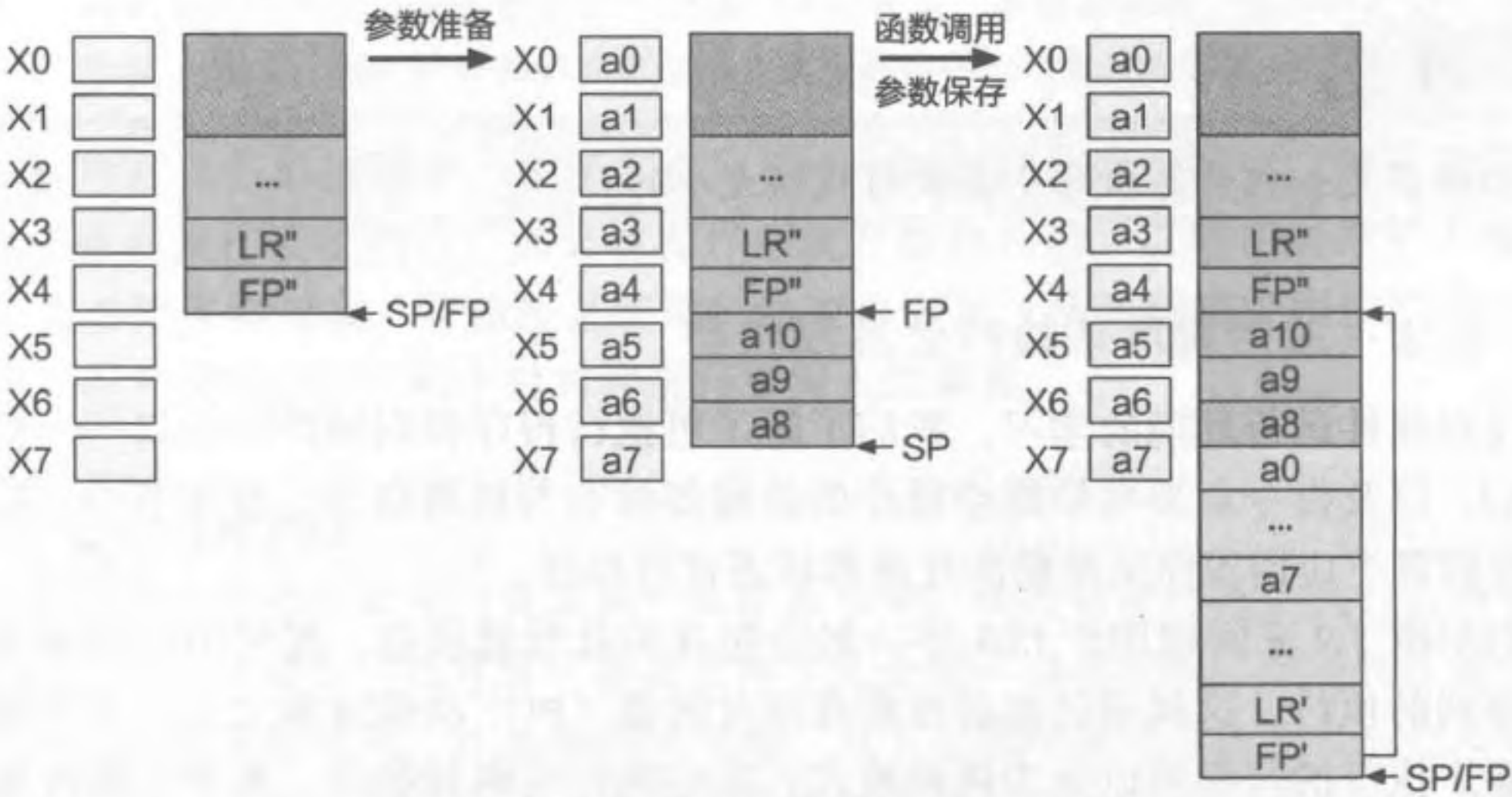


图 3.7 函数调用时的参数传递和保存过程

完整的栈帧结构

经过前面对调用惯例的学习，我们对AArch64的栈帧结构有了更全面的认识。每个函数除了在栈帧中保存FP和LR的旧值外，还会利用栈帧中的不同区域来分别存储调用时传递的实际参数、函数内部分配的局部变量、需要保存的寄存器值等数据（见图3.8）。

回想一下前面几节学习的例子，我们可以发现并非所有函数都需要栈帧。换句话说，栈帧中的每个区域都是可选的。考虑前面例子中的square函数，首先，它不包含对其他函数的调用，因此不需要担心LR与FP的值被覆盖的问题，也就不需要保存LR与FP的值；其次，它也不需要参数构造区来准备调用时需要传递的参数。X9~X15寄存器足以存放它的所有局部变量，因此用不到需要被调用者保存的寄存器，也就不需要寄存器保存区和局部变量区。它的形式参数在生命周期内仅被使用一次，因此不需要参数保存区。这样一来，对于这个函数来说栈帧中的每个部分都没有存在的必要，编译器也就不会为它分配栈帧了。

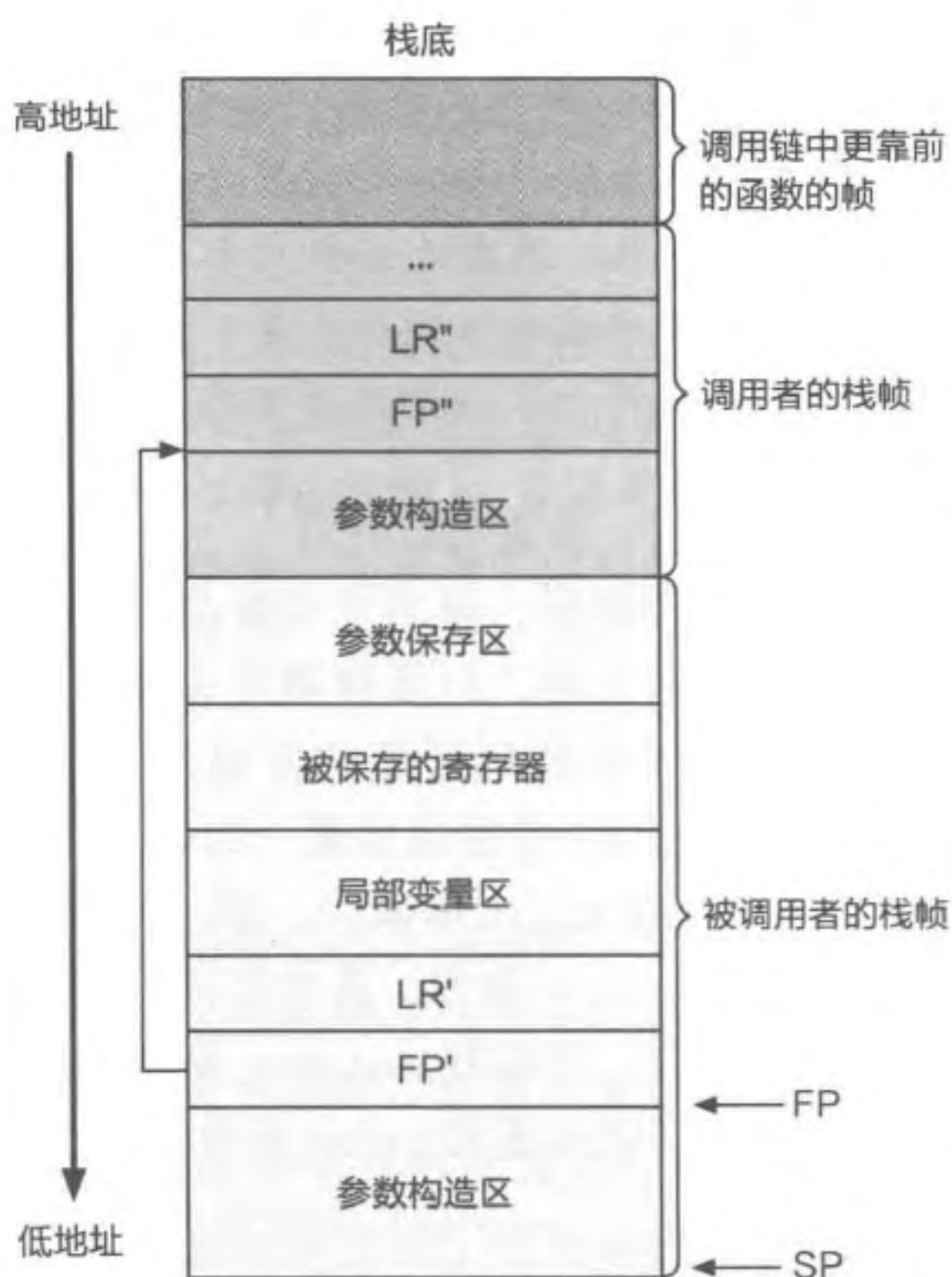


图 3.8 AArch64 架构中栈帧的结构

练习

3.7 编译器是如何计算出每个函数的栈帧大小的？

3.1.6 小结：应用程序依赖的处理器状态

经过对硬件运行环境的学习，我们了解了可执行程序控制硬件的接口——指令集架构（ISA），以及指令集架构暴露给程序的处理器状态与机器指令。在本节中，我们将对之前学习的各类应用程序所依赖的处理器状态进行总结。

我们用图3.9来回顾用户ISA中一般会包含的处理器状态。程序的运行本质上是机器指令序列的执行，这期间处理器根据程序计数器（PC）的值来确定下一条要执行的指令的位置。执行的过程可以分为两种模式：顺序执行与跳转执行。其中，跳转执行往往用于实现分支、循环等控制结构，这要依靠存储在状态寄存器中的条件码来记录上一条

指令的执行状态，从而对程序中定义的条件进行判断，以决定接下来是否进行跳转。在进行运算时，程序需要先将数据从内存加载到通用寄存器中，运算完成后再将结果写回内存。在进行函数调用时，调用者可通过通用寄存器和内存将参数传递给被调用者；被调用者在运行时所使用的本地变量以及函数返回的地址被保存在运行时栈中，处理器通过栈寄存器来保存当前栈顶的位置。

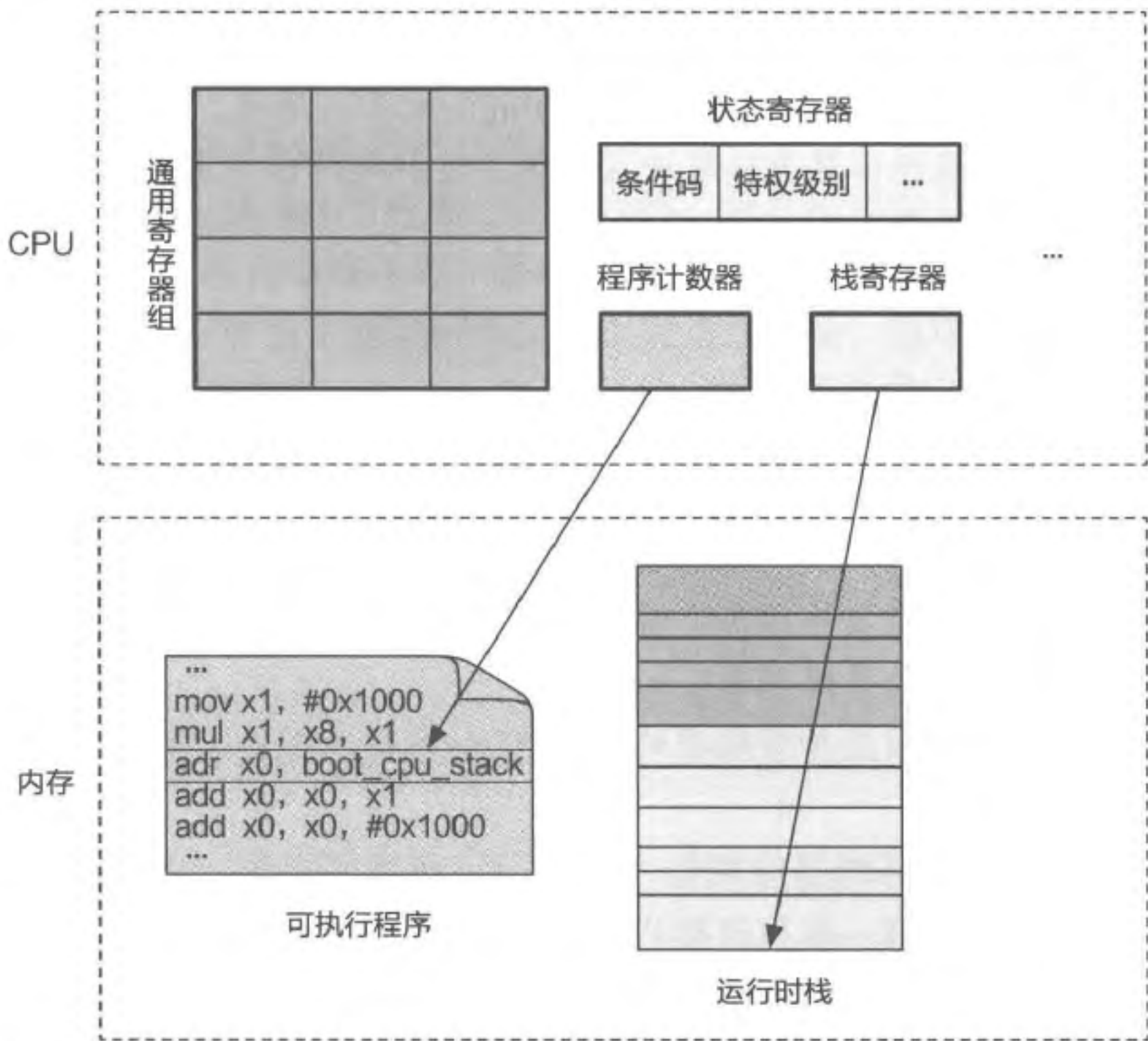


图 3.9 应用程序依赖的处理器状态，包括通用寄存器组、状态寄存器、程序计数器和栈寄存器。其中，程序计数器和栈寄存器分别存放内存中机器指令和运行时栈的地址

应用程序在运行的过程中，由于某些原因（比如时间片运行结束，需要访问硬件设备等）会切换到操作系统执行，那么在操作系统开始执行前，这些在处理器中属于应用程序的状态必须保存起来，等操作系统运行结束再恢复，从而继续应用程序的运行。我们将会在下一节探讨进程的概念时对此进行更深入的阐释。

小知识：栈的增长方向

虽然运行时栈的增长方向因架构和编译器而异，但大部分情况下我们见到的栈都从高地址向低地址增长，这种设计其实是为了充分利用内存空间。应用程序能够动态分配的内存主要分为两类：栈内存和堆内存。为了让栈和堆可以共享同一个大的内存池，操作系统往往会让栈与堆从一块内存区域的两端向中间增长。其中，堆从低地址

向高地址增长，而栈从高地址向低地址增长。

这样的设计事实上带来了严重的安全问题。如果应用程序在栈上分配了一块缓冲区并向其写入数据，一旦写入的数据量超过了缓冲区的大小，超出的部分就会覆盖栈上的其他数据，即缓冲区溢出（Buffer Overflow）。由于栈的增长方向是从高到低的，与缓冲区溢出的方向相反，因此攻击者可以通过溢出来覆盖返回地址（因为当前函数的返回地址会被保存在比缓冲区更高的地址），使函数在返回时跳转到恶意代码处，实现控制流劫持（Return-Oriented Programming）攻击。反之，如果栈的增长方向是从低到高的，返回地址的位置会比缓冲区更低，这样缓冲区就算溢出也无法覆盖返回地址的值，使攻击的难度大大增加。

3.2 操作系统的硬件运行环境

本节主要知识点

- 为什么要区分用户态与内核态？两者的运行环境有什么区别？
- 什么是中断、异常和系统调用？
- 用户态和内核态之间是如何切换的？CPU 和内核分别做了哪些操作？

操作系统的硬件运行环境是应用程序硬件运行环境的超集：除了运算、访存、函数调用等基本功能外，还包括一些只有操作系统才有权限执行的功能，比如对 CPU 中断的开关、对频率的调整、对内存的配置、对设备的操作等。这些功能之所以只允许操作系统使用，是因为它们会影响系统全局的状态而不仅仅是某一个应用程序的状态。为了区分操作系统与应用程序的不同运行环境，现代 CPU 通常会提供不同的特权级，让用户程序运行在低特权级，操作系统运行在更高的特权级，并限制只有高特权级才能执行特权操作。同时，CPU 也提供了在低特权级与高特权级之间的切换机制。

3.2.1 特权级别与系统 ISA

设想一下，如果不区分应用程序和操作系统运行的级别，换句话说，就是允许应用程序和操作系统一样能够使用硬件提供的所有功能、控制整个计算机，会发生什么呢？让我们考虑有多个应用程序同时运行的情况。在这些应用程序彼此配合良好且运行正常的情况下，即使不区分特权级别，系统也是可以正常运行的；但如果应用程序出现错误，则有可能影响其他应用甚至整个系统。例如，某个应用程序错误地运行了 CPU 的重置指令（reset），导致所有应用的状态丢失；两个应用配合失误，同时向内存或磁盘的同一个位置写入数据，则会导致其中一份数据的丢失。此外，由于应用程序有权限访问整个